

Real-Time Sketch-to-Geometry Preview Tool - Implementation Guide

Goal

Build a React + Vite application that allows users to draw freehand strokes on a canvas. As the user draws, the application should recognize shapes and render a clean geometric overlay in real time. The experience should feel responsive and smooth.

Recommended Stack

Frontend: React + Vite Canvas: HTML5 Canvas State Management: React hooks Deployment: Vercel Backend: Not required for the prototype. Keep recognition client-side.

Core Features

1. Drawing canvas using pointer events. 2. Capture strokes as arrays of points. 3. Live shape recognition. 4. Overlay recognized geometry. 5. Support line, rectangle, circle, arc, polygon. 6. Real-time updates. 7. Clear canvas button. 8. Undo last stroke.

Data Model

Store each stroke as: stroke_id timestamp points: [[x, y, time]] Store recognized shape: shape_type geometry_data linked_stroke_id

Recognition Strategy

Use a rule-based approach. Line: - Measure deviation from a straight line. Circle: - Closed shape. - Width and height of bounding box similar. - Radius consistency. Rectangle: - Closed shape. - Four dominant corners. Polygon: - Three or more corners. Arc: - Curved but not closed.

Live Binding

Whenever a stroke is created, edited, removed, or redrawn: 1. Recalculate geometry. 2. Replace previous interpretation. 3. Re-render immediately. No submit button should exist.

Modern UI Requirements

Use a clean SaaS-style interface. Layout: - Large drawing canvas. - Toolbar at top. - Status indicator. - Shape recognition badge. Recommended: - Rounded corners. - Subtle shadows. - Smooth animations. - Responsive layout. - Light theme with professional styling. Focus on usability rather than flashy visuals.

Performance Requirements

Use requestAnimationFrame where appropriate. Avoid unnecessary React re-renders. Recognize shapes after stroke completion. Maintain smooth drawing experience.

README Expectations

Include: - Setup instructions. - Architecture overview. - Shape recognition explanation. - Trade-offs. - Known limitations. - Future improvements.

Part 2 Guidance

Design telemetry around events rather than final drawings. Capture: - Stroke creation. - Stroke modification. - Stroke deletion. - Undo actions. - Timing information. - Stylus pressure when available. Use an event-stream approach so future ML models can learn user intent and sketching behavior.

Suggested Development Plan

Phase 1: Canvas and drawing. Phase 2: Stroke storage. Phase 3: Shape recognition. Phase 4: Geometry overlay. Phase 5: UI improvements. Phase 6: README and deployment.